ML SOCIETY

Why we split our data

The score your model gets on its own training data, and the score that actually counts.



train · 80%

validation · 20%

We trained a model to predict happy airline passengers.
Then we tested it on the **same passengers** it learned from.

Score: 1.00

A perfect score. Should we trust it?

Illustrative example · Scores: 1.0 = perfect · 0.5 = random guessing

THE PROBLEM

It just memorised the answers

1.00

on passengers it **had already seen**

0.64

on **new** passengers

Like a student who memorised last year's exam paper: perfect on that paper, lost on a new one. This is called **overfitting**.

Illustrative example: a decision tree on a simulated airline-satisfaction dataset.

KEY WORDS

Three kinds of data, like exam prep

Training data

The textbook and practice questions. The model **learns** from these.

Validation data

A mock exam. Used to **check progress** and pick the best model.

Test data

The real exam. Used **once**, at the very end.

The model must never see the mock or real exam questions while it's studying.

THE FIX

Hide some data from the model



Train on the blue part. Score on the cyan part. Because the model never saw the cyan rows, that score is **honest**.

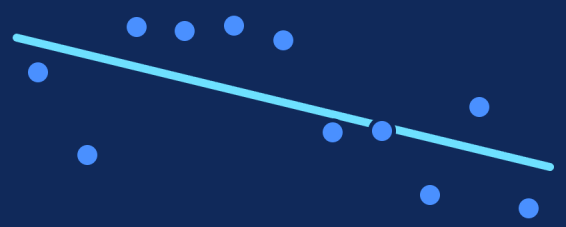
In Python it's one line:

```
train_test_split(X, y, test_size=0.2)
```

scikit-learn calls the hidden part "test". Today we're using it as our check set.

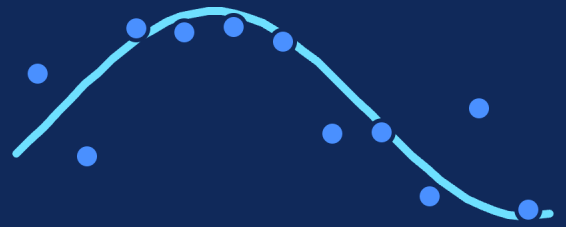
WHAT OVERFITTING LOOKS LIKE

Same dots, three models



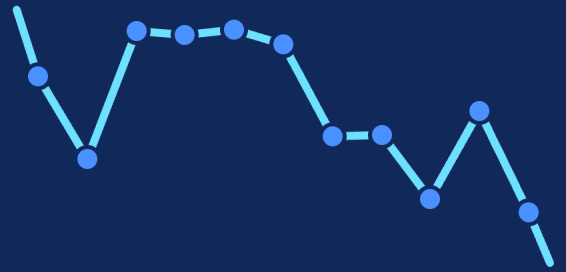
Too simple

A straight line. Misses the pattern.



Just right

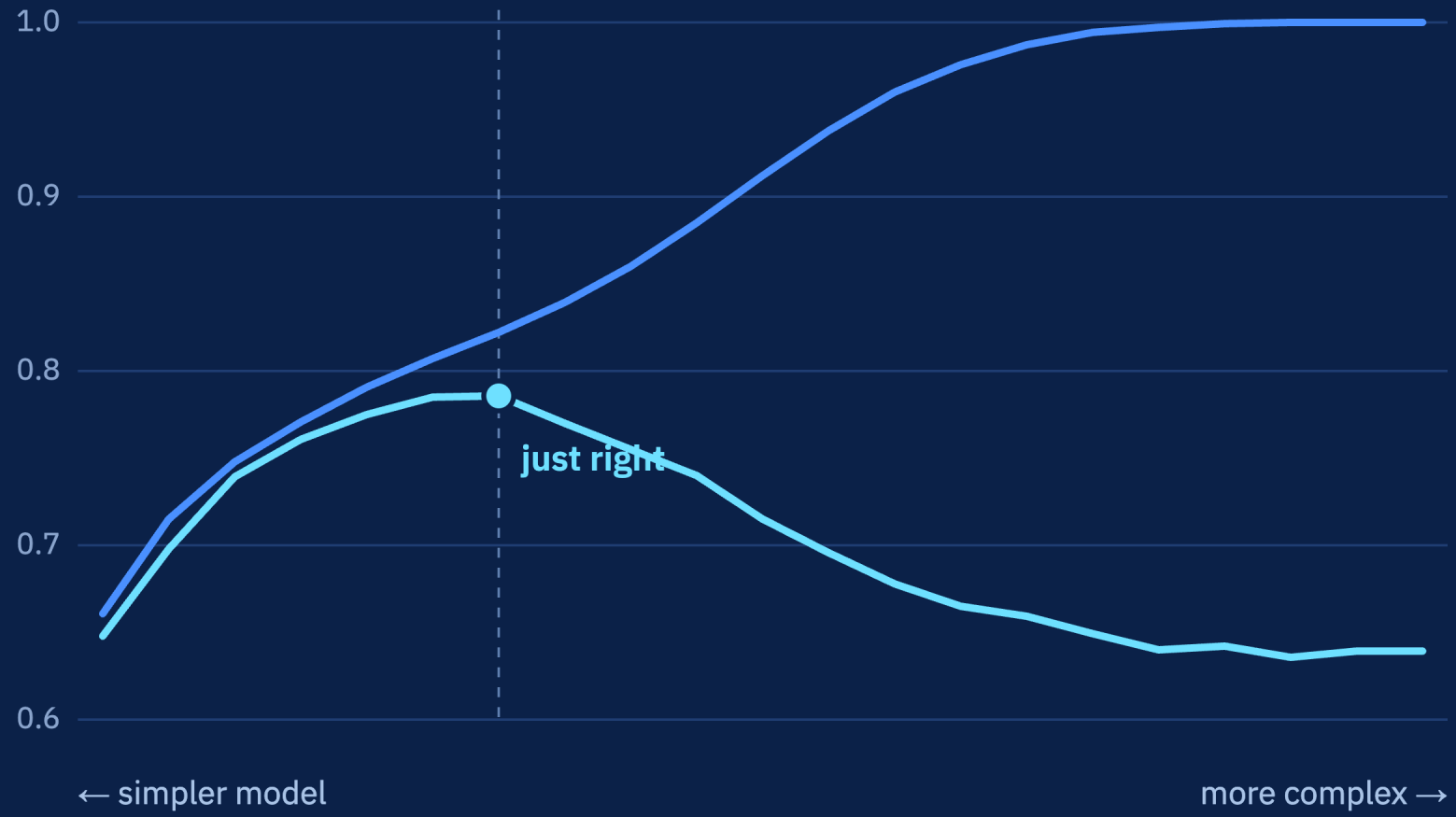
A smooth curve. Learns the pattern.



Memorised

Hits every dot, noise and all. Fails on new dots.

Too simple, too complex, just right



Score on data it's seen

Score on new data

Too simple: hasn't learned enough

Too complex: memorises instead of learning

Only the cyan line can show you where to stop.

ON KAGGLE

Kaggle uses the same idea

Exam prep	Kaggle
Textbook and practice questions	train.csv
Mock exam	Public leaderboard
Real exam	Private leaderboard

⚠ If you only practise for the mock exam, the real one will surprise you. **Trust your own check, not the public leaderboard.**

LEAKAGE

Don’t let the answers leak in

We gave a model **random noise**: nothing real to learn.
A fair test should score about 0.5, just guessing.

LEAKY

0.92

Prepared the data while looking at **all** of it, check data included. Looks brilliant. It's fake.

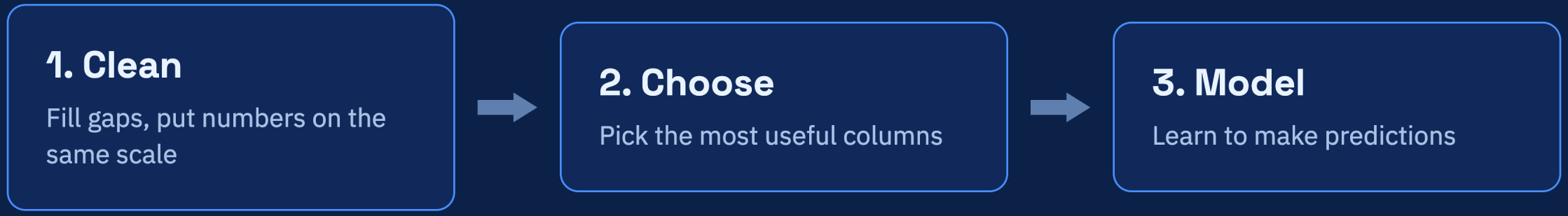
FAIR

0.54

Prepared the data using **only** the training part. Correctly says: nothing here.

HOW TO AVOID LEAKS

Bundle every step into a Pipeline

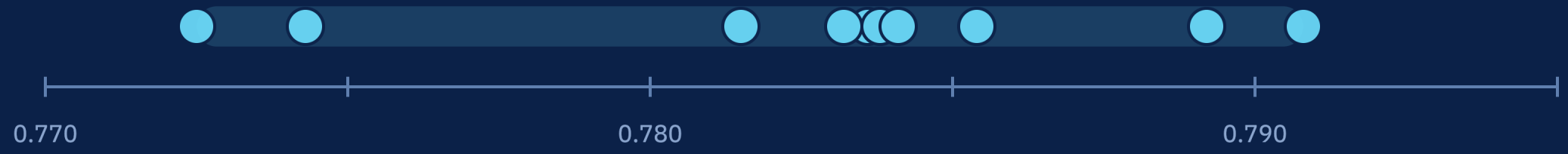


A Pipeline makes sure **every** step learns from the training data only. The check data stays hidden until scoring.

ONE MORE PROBLEM

One split can get lucky

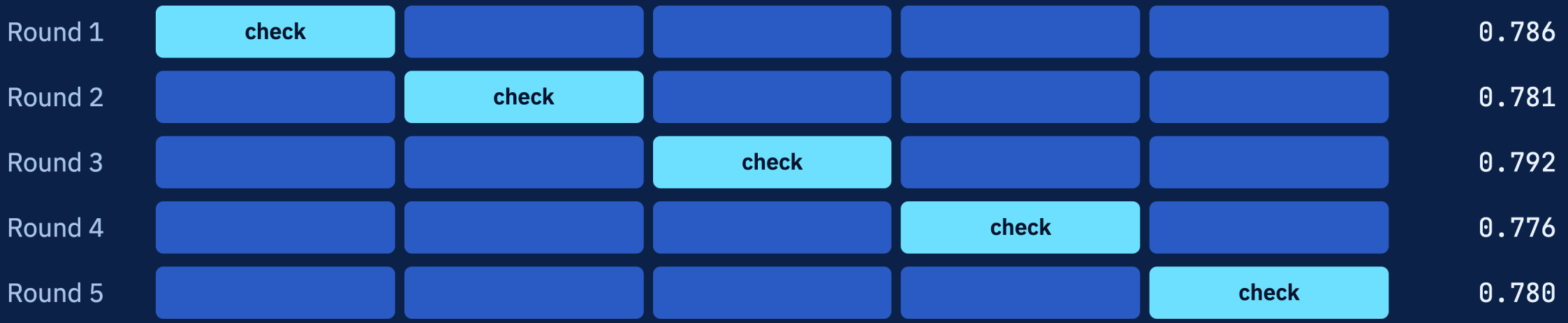
Same model, same data. We just reshuffled **which rows** went into the check set, 10 times. Each dot is one score:



The score moved just from **luck**. One split can make a model look better or worse than it really is.

CROSS-VALIDATION

Fix: take turns, then average



Average score: 0.783

CHECK YOURSELF

What would you do?

1

Your model scores 0.99 on its training data. Is it a great model?

2

You clean all the data first, then split it. Any problem?

3

You're top of the public leaderboard. Will you win?

RECAP

Five things to remember

1. A score on training data doesn't mean much
2. Always hide some data to check the model on
3. Too complex a model memorises instead of learning
4. Don't let check data leak in: use a Pipeline
5. Take turns and average: cross-validation

WHAT'S NEXT

Try it this month



Join this month's Kaggle Playground competition

kaggle.com → Competitions → Playground

Run the starter notebook: split + cross-validation